



INTRODUCTION TO PYTHON



AI-NLP-ML LAB
Department of Computer Science Engineering
IIT PATNA

Presented By:
Shrikant Malviya (s.kant.malviya@gmail.com)

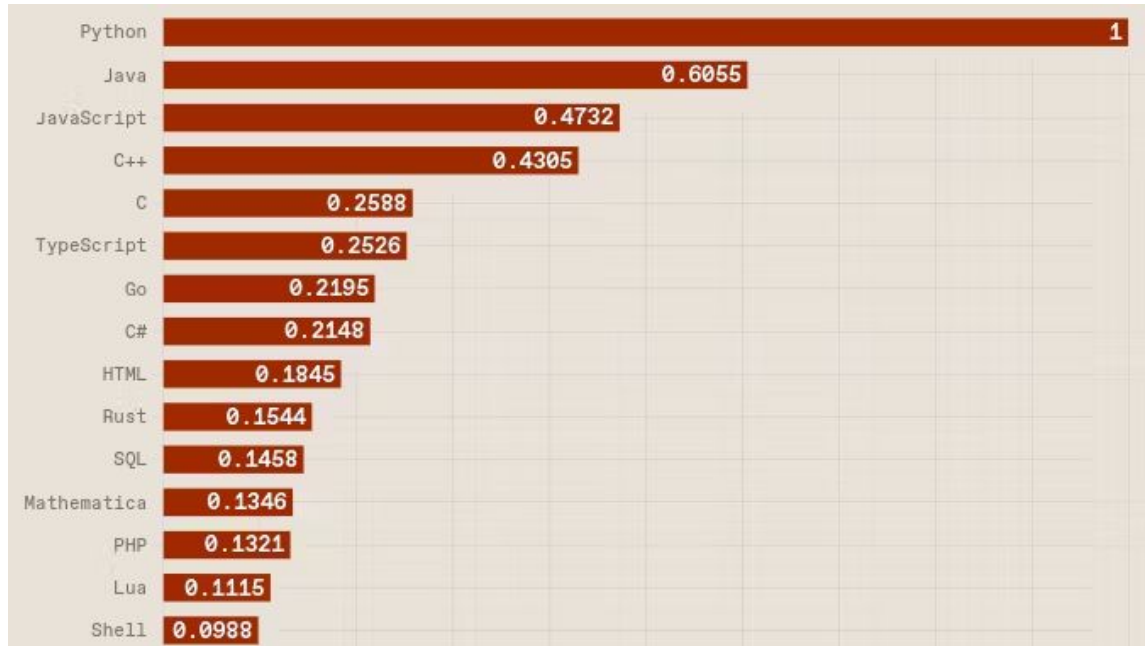
Beginning with Python programming

About Python



- a popular programming language.
- created by Guido van Rossum, and released in 1991.
- *a vocabulary ("if", "for", "print") and set of grammatical rules (syntax) for instructing a computer to perform tasks.*

About Python



IEEE list for top programming language (2024)

What can Python do?



- used on a server to create web applications (e.g. Django and Flask frameworks).
- used alongside software to create workflows (e.g. TensorFlow/PyTorch, BeautifulSoup/Scrapy).
- connect to database systems as well as can read and modify files (e.g. MySQL, File Reading/Writing).
- used to handle big data and perform complex mathematics (e.g. Pandas, Dask, NumPy, SciPy).
- used for rapid prototyping, or for production-ready software development.

Python use - Some Examples

- YouTube's original creators used Python for the most part!
- Dropbox is primarily written in Python.
- Instagram's entire backend and website are also written in Python
- Python is extremely popular in a quickly growing field of expertise called *data science* .
- hundreds of thousands of *external packages* contributed by the enormous community

About Python



- works on different platforms (Windows, Mac, Linux, Raspberry Pi, etc).
- designed for readability with the use of Indentation.
- has a simple syntax similar to the English language with influence from mathematics.
- Two main flavours of Python:
 - Python 2.7 (a “static” older version)
 - Python 3+ (mostly identical to Python 2.7 just designed to better incorporate new features)
- We will work with Python 3+

Downloading and Installing



- On Linux, it should normally be installed by default, else use the package manager
- For MacOS and Windows, download and install from <https://www.python.org/downloads/release/python-350/>
- Python using Anaconda, which provides CLI and GUI to manage and run Python environments.
<https://www.anaconda.com/docs/getting-started/getting-started>

Interpreters vs compilers



- Programming languages are “*high level*”, for humans to understand
- Computers need “*lower level*” instructions
- **Compiler:** Translates high level programming language to machine level instructions, generates “*executable*” code in “One-Go”. Ex: C/C++
- **Interpreter** : Itself a program that runs and directly “understands” high level programming language and executes the source code “Line-By-Line”. Ex: Python

Python interpreter



- Python is basically an interpreted language
 - Load the Python interpreter
 - Send Python commands to the interpreter to be executed
 - Easy to interactively explore language features
 - Can load complex programs from files
 - `>>> from filename import *`

A typical Python program



```
def function_1(...):
```

```
...
```

```
def function_2(...):
```

```
...
```

```
:
```

```
def function_k(...):
```

```
...
```

```
statement_1
```

```
statement_2
```

```
:
```

```
statement_n
```

- ❖ Interpreter executes statements from top to bottom
- ❖ Function definitions are “digested” for future use
- ❖ Actual computation starts from `statement_1`

Some resources



- The online Python tutorial is a good place to start:
<https://docs.python.org/3/tutorial/index.html>
- Here are some books, again available online:
 - Dive into Python 3, Mark Pilgrim
<http://www.diveintopython3.net/>
 - Think Python, 2nd Edition, Allen B. Downey
<http://greenteapress.com/wp/think-python-2e/>

Learning programming



- cannot be learnt theoretically.
- must write and execute your code.
- syntax is light and is relatively easy to learn.
- **Let's Go for it!**

Exploring and Learning Python

OUTLINE



- Your first program
- Data types
- Expressions and Variables
- Lists and Tuples
- String Operations

Practical Demo



First Program

```
# print("Hello World")
```

```
print("Hello World")
```

- Error Type
 - **A Syntactic error** is when Python does not understand your code. For example, if you spell print "frint", you will get an error message.
 - **A Semantic error** is when your logic is wrong. For example, if you enter "Hello Duniya" instead of "Hello World", you don't get an error message, but your code is wrong.

Python Keywords and Identifiers

- **Keywords** are the reserved words.
 - cannot use as a variable name, function name or any other identifier.
 - used to define the syntax and structure.
 - are case-sensitive.
- 35 keywords in Python 3.11. Can vary slightly over the course of time.
- For overview - [list of all the keywords](#) with examples.

Python Identifiers

- **Identifier** is a name given to entities like class, functions, variables, etc.
 - helps to differentiate one entity from another.
 - can be a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore `_`. Names like `myClass`, `var_1` and `print_this_to_screen`, all are valid example.
 - cannot start with a digit. **1variable** is invalid, but **variable1** is a valid name.
 - Keywords cannot be used as identifiers. Ex: `global = 1` O/P: `SyntaxError: invalid syntax`
 - cannot use special symbols like `!`, `@`, `#`, `$`, `%` etc. in our identifier. Ex: `a@ = 0` O/P: `SyntaxError: invalid syntax`
 - can be of any length.

Python Statement, and Indentation

1. **Statement:** Instructions that a Python interpreter can execute are called statements. For example, `a = 1` is an assignment statement.
2. **Multi-line statement:** In Python, the end of a statement is marked by a newline character. But we can make a statement extend over multiple lines with the line continuation character (`\`). For example:

```
a = 1 + 2 + 3 + \
```

```
4 + 5 + 6 + \
```

```
7 + 8 + 9
```

3. Indentation:

- A code block (body of a function, loop, etc.) starts with indentation and ends with the first unindented line. The amount of indentation is up to you, but it must be consistent throughout that block.

```
for i in range(1,11):
```

```
    print(i)
```

```
    if i == 5:
```

```
        break
```

TYPES

Types

- represents different types of data.
- Python Numbers: Integers, floating point numbers and complex numbers fall under Python numbers category.
 - Int
 - Float
 - Complex number

NOTE:

We can use the `type()` function to know which class a variable or a value belongs to.

Similarly, the `isinstance()` function is used to check if an object belongs to a particular class.

The first column indicates the expression. The second column indicates the data type.

<code>type(11)</code>	<code>int</code>
<code>type(11.11)</code>	<code>float</code>
<code>type("Hello World")</code>	<code>str</code>

- Integers can be negative or positive. It should be noted that there is a finite range of integers, but it is quite large.
- Floats are real numbers; they include the integers but also numbers in between the integers.

Python Type Conversion and Type Casting

The process of converting the value of one data type (integer, string, float, etc.) to another data type is called type conversion. Python has two types of type conversion.

1. Implicit Type Conversion
2. Explicit Type Conversion (type casting)

Implicit Type Conversion

In Implicit type conversion, Python automatically converts one data type to another data type. This process doesn't need any user involvement.

Example 1: Converting integer to float

```
num_int = 123
```

```
num_flo = 1.23
```

```
num_new = num_int + num_flo
```

```
print("datatype of num_int:",type(num_int))
```

```
print("datatype of num_flo:",type(num_flo))
```

```
print("Value of num_new:",num_new)
```

```
print("datatype of num_new:",type(num_new))
```

Explicit Type Conversion

- In Explicit Type Conversion, users convert the data type of an object to required data type. We use the predefined functions like `int()`, `float()`, `str()`, etc to perform explicit type conversion.
- This type of conversion is also called **typecasting** because the user casts (changes) the data type of the objects.

Syntax :

`<required_datatype>(expression)`

Typecasting can be done by assigning the required data type function to the expression.

Expressions and Variables

Expressions

- describe a type of operation that computers perform.
- are operations that Python performs.
 - For example, basic arithmetic operations like adding multiple numbers.

Example:

1. $60 + 40 + 25$ (operand: 60,40,20 and operator: +)
2. 25 divided by 5 ($25/5$) is 5. 25 divided by 6 ($25/6$) is approximately 4.167.
3. We can use the double slash for integer division, where the result is rounded. ($25 // 6 = 4$)

Variables

- is a named location used to store data in the memory. It is helpful to think of variables as a container that holds data that can be changed later in the program.
- For example, `number = 10`

Here, we have created a variable named `number`. We have assigned the value 10 to the variable.

```
number = 10
```

```
number = 1.1
```

Initially, the value of `number` was 10. Later, it was changed to 1.1.

Assigning values to Variables in Python

you can use the assignment operator = to assign a value to a variable.

Example 1: Declaring and assigning value to a variable

```
website = "apple.com"
```

```
print(website)
```

NOTE:

Python is a type-inferred language, so you don't have to explicitly define the variable type. It automatically knows that apple.com is a string and declares the website variable as a string.

cont..

Example 2: Changing the value of a variable

```
website = "apple.com"
```

```
print(website)
```

```
# assigning a new value to website
```

```
website = "programiz.com"
```

```
print(website)
```

In the above program, we have assigned apple.com to the website variable initially. Then, the value is changed to programiz.com.

Example 3: Assigning multiple values to multiple variables

```
a, b, c = 5, 3.2, "Hello"
```

```
x = y = z = "same"
```

```
print (a)
```

```
print(x)
```

```
print (b)
```

```
print(y)
```

```
print (c)
```

```
print(z)
```

Constants

- A constant is a type of variable whose value cannot be changed. It is helpful to think of constants as containers that hold information which cannot be changed later.
- You can think of constants as a bag to store some books which cannot be replaced once placed inside the bag.

Assigning value to constant in Python

Constants are usually declared and assigned in a module. Here, the module is a new file containing variables, functions, etc which is imported to the main file. Inside the module, constants are written in all capital letters and underscores separating the words.

Example 3: Declaring and assigning value to a constant

Create a constant.py:

```
PI = 3.14
```

```
GRAVITY = 9.8
```

Create a main.py:

```
import constant
```

```
print(constant.PI)
```

```
print(constant.GRAVITY)
```

Rules and Naming Convention for Variables and constants

1. Constant and variable names should have a combination of letters in lowercase (a to z) or uppercase (A to Z) or digits (0 to 9) or an underscore (_). For example: snake_case, MACRO_CASE, camelCase, CapWords
2. Create a name that makes sense. For example, vowel makes more sense than v.
3. If you want to create a variable name having two words, use underscore to separate them. For example: my_name, current_salary
4. Use capital letters possible to declare a constant. For example: PI, G, MASS, SPEED_OF_LIGHT, TEMP
5. Never use special symbols like !, @, #, \$, %, etc.
6. Don't start a variable name with a digit.

Lists and Tuples

Tuples

- lists and tuples, these are called compound data types and are one of the key types of data structures in Python
- **Tuples are an ordered sequence**
- Tuples are expressed as **comma-separated elements within parentheses**

Ratings = (10,9,6,5,10,5,4)

- In python, we have different data types:
 - 'Disco': str
 - 12 : int
 - 4.5 : float

Note:

They can all be contained in a tuple, but the type of the variable is tuple.

tuple1=('disco',12,4.5)

type(tuple1) o/p: tuple

	index_no	element	
-3	0	disco	tuple1[0]='disco'
-2	1	12	tuple1[1]=12
-1	2	4.5	tuple1[2]=4.5

Tuple Concatenation

- We can concatenate or combine tuples by adding them.

```
tuple1 = ('disco', 12, 4.5)
```

0	1	2
---	---	---

```
tuple2 = tuple1 + ("hard rock",10)
```

```
('disco', 12, 4.5,"hard rock",10)
```

0	1	2	3	4
---	---	---	---	---

Tuple Slicing

```
tuple2 = ('disco', 12, 4.5, "hard rock", 10)
```

0	1	2	3	4
---	---	---	---	---

```
tuple2[0:3] = ('disco', 12, 4.5)
```

```
tuple2[3:5] = ('hard rock', 10)
```

Notice how the last index is 1 larger than the length of the tuple.

Tuple Length

- Use len() function

```
tuple2 = ('disco', 12, 4.5, "hard rock", 10)
```

```
len(tuple2) = 5
```

```
Ratings = (10,9,6,5,10,5,4)
```

```
len(Ratings) = ??
```

Tuples Immutable

- Tuples are immutable, which means we can't change them.

Ratings = (10,9,6,5,10,5,4)

Ratings[2] = 4 (It will show error)

TypeError: 'tuple' object does not support item assignment

- If we would like to manipulate a tuple, we must create a new tuple instead.
- For example, if we would like to sort a tuple, we use the **function sorted**.

SortedRating= sorted(Ratings)

Tuples: Nesting

- A tuple can contain other tuples as well as other complex data types; this is called nesting.
- We can access these elements using the standard indexing methods.

NT = (1, 2, ('pop', 'rock'), (3, 4), ('disco', (1, 22)))

0	1	2	3	4
---	---	---	---	---

NT[0] = 1

NT[1] = 2

NT[2] = ('pop', 'rock')

0	1
---	---

NT[3] = (3, 4)

0	1
---	---

NT[4] = ('disco', (1, 22))

0	1
---	---

NT[2][1] = rock

NT[3][0] = 3

NT[4][1] = (1, 22)

Question: How can we get value 22? NT[4][1][1]

NT = (1, 2, ("pop", "rock"), (3, 4), ("disco", (1, 2)))



NT[2]

NT[3]

NT[4]

("pop", "rock")

(3, 4)



("disco", (1, 2))

"pop"

"rock"

3

4

"disco"

(1, 2)

NT[2][0]

NT[2][1]

NT[3][0]

NT[3][1]

NT[4][0]

NT[4][1]

("pop", "rock")

(3,4)

("disco", (1,2))

"pop"

"rock"

3

4

"disco"

(1,2)

NT[2][0]

NT[2][1]

NT[3][0]

NT[3][1]

NT[4][0]

NT[4][1]

NT[2][1][0]

NT[2][1][1]

NT[4][1][0]

NT[4][1][1]

r

o

1

2

Lists

- List is an ordered sequence of items. It is one of the most used data type in Python and is very flexible.
- All the items in a list do not need to be of the same type.
- Declaring a list is pretty straight forward. Items separated by commas are enclosed within brackets [].

```
a = [1, 2.2, 'python']
```

NOTE:

In many respects lists are like tuples, one key difference is they(lists) are **mutable**.

Lists

- Lists can contain strings, floats, integers, nested lists, nested tuples and other data structures
- The same indexing conventions apply for nesting.
- Like tuples, each element of a list can be accessed via an index.

`["Michael Jackson", 10.1, 1982, [1,2], ('A',1)]`

Lists: access

L = ["Michael Jackson", 10.1, 1982]

-3
-2
-1

0	"Michael Jackson"
1	10.1
2	1982

L[0]

L[1]

L[2] = L[-1]

List: Slicing

L = ["Michael Jackson", 10.1, 1982, "MJ", 1]

0	1	2	3	4
---	---	---	---	---

L[3:5] = ["MJ", 1]

NOTE

The index conventions for lists and tuples are identical.

List concatenate

$L = [\text{"Michael Jackson"}, 10.1, 1982]$

$L1 = L + [\text{"pop"}, 10]$

$L1 = [\text{"Michael Jackson"}, 10.1, 1982, \text{"pop"}, 10]$

0	1	2	3	4
---	---	---	---	---

Lists are Mutable

- Lists are mutable, therefore, we can change them

```
L = ["Michael Jackson", 10.1, 1982]
```

```
L.extend(["pop",10])
```

```
L = ["Michael Jackson", 10.1, 1982, "pop",10]
```

0	1	2	3	4
---	---	---	---	---

```
L.append(["pop",10])
```

```
L = ["Michael Jackson", 10.1, 1982,["pop",10]]
```

0	1	2	3
---	---	---	---

- Every time we apply a method, the lists changes.
- If we apply extend, we add two new elements to the list. The list L is modified by adding two new elements.
- If we append the string “A“, we further change the list adding the string “A”.
- As lists are mutable, we can change them.

```
A = ["disco", 10, 1.2]
```

```
A[0] ="hard rock"
```

```
A = ["hard rock", 10, 12]
```

- We can delete an element of a list using the "del" command; we simply indicate the list item we would like to remove as an argument.

```
del(A[0])
```

String to a list

- We can convert a string to a list using split.
- For example, the method split converts every group of characters separated by a space into an element of a list.

`"hard rock".split()` output: `["hard", "rock"]`

- We can use the split function to separate strings on a specific character, known as a **delimiter**.

`"A,B,C,D".split(',')` output: `["A", "B", "C", "D"]`

Lists: Aliasing

- Multiple names referring to the same object is known as aliasing.

```
A=["hard rock", 10, 4.5]
```

```
B = A
```

- If we change the first element in "A" to "banana" we get a side effect; the value of B will change as a consequence.
- "A" and "B" are referencing the same list, therefore if we change "A", list "B" also changes.

```
B[0] = "hard rock"
```

```
A[0] = "banana"
```

```
print("list A: ",A,"\nlist B:",B)= ??
```

Lists: clone

- You can clone list “A” by using the following syntax.

```
A=["hard rock", 10, 4.5]
```

```
B = A[:]
```

- Variable "A" references one list.
- Variable “B” references a new copy or clone of the original list.
- Now if you change “A”, "B" will not change.

String Operations

Python Strings

- String is sequence of Unicode characters. Multi-line strings can be denoted using triple quotes, `'''` or `"""`.

```
s = "This is a string"
```

```
s = """A multiline
```

```
string"""
```

- a string is a sequence of characters. A string is contained within two quotes:

```
“ Michael Jackson is the best “
```

- You could also use single quotes: `‘ Michael Jackson is the best ‘`
- A string can be spaces, or digits: `“1 2 3 4 5 6”`
- A string can also be special characters: `“! @ # $ %”`

- We can bind or assign a string to another variable. It is helpful to think of a string as an ordered sequence.

Name= "Michael Jackson"

M	i	c	h	a	e	l		J	a	c	k	s	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

- Each element in the sequence can be accessed using an index represented by the array of numbers.

Name[0] = M Name[6] = l Name[13] = o

- We can also use negative indexing with strings. The last element is given by the index -1. The first element can be obtained by index -15, and so on

-15	-14	-13	-12	-11	-10	-9	-8	-7	-6	-5	-4	-3	-2	-1
-----	-----	-----	-----	-----	-----	----	----	----	----	----	----	----	----	----

String Slicing

We can bind a string to another variable. It is helpful to think of string as a list or tuple.

Name="Michael Jackson"

M	i	c	h	a	e	l		J	a	c	k	s	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Name[0:4]= Mich

Name[8:12]= Jack

Strings are immutable.

```
s = 'Hello world!
```

```
print("s[4] = ", s[4]) # s[4] = 'o'
```

```
print("s[6:11] = ", s[6:11]) # s[6:11] = 'world'
```

```
# Generates error
```

```
# Strings are immutable in Python
```

```
s[5] = 'd'
```

String Stride

- We can also input a stride value as follows. The 2 indicates we select every second variable.

M	i	c	h	a	e	l		J	a	c	k	s	o	n
0	1	2	3	4	5	6	7	8	9	10	11	12	13	14

Name[::2] = McaJcsn

- We can also incorporate slicing, for ex:

Name[0:5:2] = Mca

String Length

- Use **len()** function

`len("Michael Jackson")` output: 15

String Concatenation

- We can concatenate or combine strings. We use the addition symbols. The result is a new string that is a combination of both.

Name = "Michael Jackson"

Statement = Name + "is the best"

Statement = "Michael Jackson is the best"

- We can **replicate values of a string**. We simply multiply the string by the number of times we would like to replicate it, in this case, three.

3 * "Michael Jackson" o/p: "Michael Jackson Michael Jackson Michael Jackson"

- The result is a new string. The new string consists of three copies of the original string

Strings: escape characters

- **Backslashes** represent the beginning of escape characters.
- Escape sequences represent strings that may be difficult to input.

For example, backslashes "n" represent a new line.

```
print("Michael \n Jackson")
```

Similarly, backslash "t" represents a tab. The output is given by a tab where the backslash "t" is.

```
print("Michael \t Jackson")
```

If you want to place a backslash in your string, use a double backslash. The result is a backslash after the escape sequence. We can also place an r in front of the string.

```
print("Michael \\ Jackson")   or   print(r"Michael \ Jackson")
```

String Method

- UpperCase

A = "This is a string"

B = A.upper()

- LowerCase

A = "This Is A String"

B = A.lower()

- Replace

A = "This is a string"

B = A.replace("This", "That")

- Find: return first index of the sequence

A = "This is a string"

A.find("is")



THANK YOU